

94-eos-sys-dev-dbs • EOS 系统设计产品数据分解结构

本文定义 EOS 系统设计各阶段输出产品数据文档的**分解结构**——即以**树形结构**表达的逐层分解（分解结构 = 树形结构，两词同义混用）。下列章节为框架，各节结构树逐节填充。

版本：v1.14 | 修订：2026-09-10

一、产品数据及其形态

本文各章分别定义一份**产品数据**的分解结构（树）。本章是全篇共用的前提——EOS 有哪些产品数据、各落在哪个空间（1.1），**各树凭什么成立**（1.2 三个前提），每份产品数据有哪两个形态（1.3），形态之间怎么承接及已定的树间对应（1.4）。未定处标【待裁】，待相应树补齐后再回填。

1.1 EOS 的产品数据清单

EOS 设计线以「需求—方案结对」逐对生成产品数据；结对之前另有一条**原始需求入口加工链**，把外部材料加工成第一份需求产品数据。开发过程应输出下列产品数据——§二 逐一给出其分解结构：

产品数据	文档（EOS 实例）	空间	本文
原始需求 †	00-origin-requirement-materials/10-raw-files/（任意载体）	业务	§2.0
文本化的原始需求 †	*.textualized.md	业务	§2.0
切分后的原始需求 †	*.chunk-{seq}.md（status=raw）	业务	§2.0
澄清后的原始需求 †	同上文件（status=clarified）	业务	§2.0
相关方需求	01-eos-specified-requirements.md	业务	§2.1
业务流程概要	02-eos-business-summary-architecture.md	业务	§2.2
业务流程方案	04-eos-business-detailed.md	业务	§2.2
系统需求概要	05-eos-system-requirement-summary-architecture.md	方案	§2.3

产品数据	文档 (EOS 实例)	空间	本文
系统需求方案	06-eos-system-requirement-detailed.md	方案	§2.3
平台构成概要	07-eos-platform-product-architecture.md	方案	§2.4
平台构成方案	(文档待定义)	方案	§2.4
配置方案	(文档待定义)	方案	§2.5
前后端组件方案	(文档待定义)	方案	§2.6

名称自带形态：...概要 = 概要态、...方案 = 完整态——同一份产品数据的两个形态各占一行，同指一节。† 四行是**同一份产品数据的入口加工形态**（一份材料加工到多细的程度序列），不是四份产品数据；链条与其结构见 §2.0。两处例外：01 相关方需求不分形态；**§2.5 配置方案 / §2.6 前后端组件方案** 的名称不含形态标记，其形态归属【待裁】（注意：名称中的"方案"不表示完整态）。文档清单与命名以 91 规范 §11.1 为准（表中 相关方需求 = 91 所称「规范化的相关方需求」）；03-eos-business-summary-detailed.md 旧档型已退役（概要不含叶，见 93 规范 §三.3）。

两个空间——上表"空间"列的含义：业务空间里的树回答"谁做什么、产出什么"，先不涉及由哪套电脑系统来实现；方案空间里的树写"这套承接系统要实现什么、由什么构成"（本设计实例中该承接系统 = EOS 平台，企业运营体系的信息化形态）。

空间标签是成对相对而言的，不是全篇的属性：§2.3 相对 §2.1/§2.2 是"方案"（写系统要实现什么），相对 §2.4 又是"需求"（§2.4 写这套系统的软件构成）。"业务/方案"只说一对之内谁写问题、谁写解，不指文档等级。

§2.2 的业务树名：两个视角族 × 产品索引

每个产品有**两棵视角树**——R 树（角色履职视角，roles & responsibilities）与 P 树（产品开发视角，product or service）。下标索引的是所指产品本身：0 = 平台本身， $i \geq 1$ = 基于平台引擎构建的各个业务。

i	所指产品	R 树（角色履职视角）	P 树（产品开发视角）
0	平台本身	R ₀ 【待确认】	§2.2.3 引擎开发业务流程（构件 = 引擎，单分支只有配置单元）
$i \geq 1$	基于平台引擎构建的业务	§2.2.1 角色履职业务流程（相关方角色按职责完成业务对象闭环）	§2.2.2 产品开发业务流程（构件经 WBS 产出构件产品数据）

两族与既有视角名一一对应：R ↔ SL（角色场景视角）、P ↔ PL（输出产品视角）——见 91 §3.1 的四套视角层级表。本文不另起一套口径，改名只是把"族"从隐含变显式。**下标索引的**

是"所指产品"，与 SLO / PLO 的层级 0 (= 一切树的公共根) 不是同一回事；书面上以带下标形式 (R₀ / R_i) 与 91 的约束编号 R1~R4 (始终带 §2.5 / §11 语境) 相区分。R₀【待确认】：平台自身的角色履职树尚未独立成树——其内容现以碎片形式存在 (02 预置的平台治理角色场景业务、eng 链 bph-eng 的业务配置人员场景)。

1.2 本文各树凭什么成立：三个前提

本文全篇建立在三个前提之上。它们不是本文的规定，而是已被外部权威论证过的结论——本节给出结论与依据名称，完整论证 (引文、出处、反例) 见 95 号研究报告。

前提一·为什么产品数据可以条目化

每份产品数据都必须能拆到**条目**。理由：设计的每一步产物都要能被**独立判定**"是否被满足"，而通行的需求书写标准正是这样要求单条需求的——**单数、可验证、唯一解释** (ISO/IEC/IEEE 29148 的良构需求特性，其中 **Singular** 一名即"一条只讲一件事")。条目的判定标准 (分解终止标准) 引[通用规范 §2.2.2](#)，本文不重述。

完整论证见 95 §二。

前提二·为什么产品数据可以树形化

因为**分层是复杂系统的常态结构** (Simon 的层级与近可分解性)，且工程上早有分解纪律 (WBS 的 **100% 规则**，即 MECE)。但要注意：**"能组树"不是自动的**——同一层的分组必须同时满足三条，否则得到的不是树：

#	条件	不满足则
一	判据单维 ——同一层的分组只用一个判据 (如"系统职责")，不得混用两个 (如职责与业务域并用)	同层内出现 交叉分类 → 得到的是 格 ，不是树
二	条目单归属 ——每条只归一个父节点 (单继承)	得到的是 有向无环图 (DAG) (多层级)，不是树
三	分组完备互斥 ——子层完整覆盖父层、且互不重叠	出现 缺口或重叠 → 不是合格的树

本文各树均按此建立；下游 (数据文件「树画像」区、23、人类方案的任务视图) 按本文对齐时，同样以此自查。

完整论证见 95 §三。

前提三·为什么由需求设计方案可以 N:1 分配

因为设计被建模为**从需求域到方案域的映射** (公理化设计的 FR↔DP 映射)。这条映射的形状由两端与一条约束决定：

- **定义域取需求侧的原子（叶）**——粗粒度节点**不能被 1:1 分配**：它内部含着多条性质不同的需求，交出去后无法判定谁负责哪一部分；
- **值域取方案侧当前可负责的实体（枝梢）**——这一刻**方案侧的叶尚未产出**（方案只到概要），方案侧能提供的最深实体就是架构末级节点；
- **映射必须是函数**——函数允许多对一（N:1：一个实体承担一族需求 = 内聚），**不允许一对多**（1:N：一条需求散在多个实体 = 变更传播与重复实现）。注意**两侧理由不同源**。

落地于 §1.4 的承接规则。

完整论证见 95 §四。

1.3 产品数据的两个形态

每份产品数据都是**同一棵树**，有**概要态**与**完整态**两个形态。四个级语义术语（全树通用）：

术语	含义
叶 / 叶节点	树的 末级节点
枝梢 / 枝梢节点	叶的 上一级节点 ——概要的最深节点
概要（态）	根 → 枝梢 （不含叶节点）
完整态	根 → 叶 （含叶节点）

叶只在其**完整态**产出，概要到枝梢为止。

叶不只是“最深的那一级”——它有判据：**叶必须切到可被独立解释、独立验证、可被唯一交出去的粒度**。所以“某棵树到哪一级为止”是一个**判断**，不是一个位置约定；判定标准（分解终止标准）见**通用规范 §2.2.2**，本文不重述。

1.4 形态之间的承接与已定的树间对应

承接规则：上一份产品数据的**最深可用原子（叶）**，对到下一份产品数据**能负责这条语义的节点——枝干或枝梢，以语义定级**；分配关系不变。

- **两形态之间不是承接**：同一份产品数据的概要态与完整态之间（02↔04、05↔06）属 §1.3 的形态轴，不在此列。本链上的承接只有三处：01→02、04→05、06→07。
- **基数：可 N:1，不可 1:N**——多条上游叶节点可汇入同一个下游节点；一条上游叶节点只能被一个下游节点承接。
- **依据**：为什么取“最深可用原子”、为什么是 N:1——见 §1.2 前提三。
- 已定的**承接**对应：

承接	上游叶	下游落点
01 → 02	需求条目 (§2.1.1 ③)	§2.2.1 的 ④ 场景业务 (枝干) ——一条需求归入它所描述的那个场景
04 → 05	§2.2.1 / §2.2.2 的叶 (R _i / P _i 两棵业务树末级同义; P _o 同此理, 其叶级见 §2.2.3 【待裁】)	§2.3.1 的 枝梢 (⑤ 用例场景功能)

"叶 → 枝梢"只是其中一例的显影: 在"上游已到完整态、下游只到概要"的步上 (04→05), 落点恰是枝梢、于是错开一级。这只是两侧"**各自最深可用原子**"对齐的一般结果在特定完成度差下的样子——**不推出**"上游第 N 级对应下游第 N-1 / N-2 级"这类数量关系, 也不表示落点必在枝梢 (01→02 就落在枝干)。

其余已定的树间对应 (非承接——同一对象的两面或两态, 不涉及视角转换):

对应双方	关系
§2.3.1 ① 信息系统 ↔ §2.4 ① 信息系统	同一系统的两面 : 需求侧功能分解宿主 = 软件构成侧 (模型/前后端组件) 宿主; 本设计实例 = EOS 平台 (企业运营体系的信息化形态)
§2.4 根 企业信息系统	§2.3.1 ① 之上的 语境容器 ——企业整体信息系统, EOS 为其一
§2.1.2 ↔ §2.3.2	同一 NFR 分类体系的两态: 相关方侧 NFR (定性/待澄清目标值) → 系统侧 SysReq-NFR (量化+约束); §2.3.2 与 §2.3.1 共用 ① 信息系统 根

【待裁】§2.3.1 的叶 (⑥ IPO功能) 由 §2.4 的哪一级承接, 待 §2.4 结构树补齐后回填。

二、各产品数据的分解结构

本节逐份给出 §1.1 清单表所列各份产品数据的结构树。级语义 (叶 / 枝梢 / 概要态 / 完整态)、成立前提与树间承接见 §一。

2.0 原始需求 (入口加工链)

原始需求是设计线的**入口产品数据**——外部材料在此被加工成 §2.1 的相关方需求基线。本节给这条链的**加工程度序列** (2.0.1) 与链上**长出的结构** (2.0.2); §2.1 给终点 (相关方需求) 的完整树。

2.0.1 五个加工形态

加工形态	载体（EOS 实例）	产出工序	结构上做了什么
原始需求	00-origin-requirement-materials/10-raw-files/（PDF/图片/邮件/纪要/已有 .md）	外部输入	—（尚在 EOS 结构之外）
文本化的原始需求	*.textualized.md	eos-ort00-textualize	只换载体，结构照搬原文
切分后的原始需求	*.chunk-{seq}.md（status=raw）	eos-ort01-chunk	长出第一级 ——按业务对象闭环切
澄清后的原始需求	同上文件（status=clarified）	eos-ort02-clarify	树形不动，只消歧
规范化的相关方需求	01-eos-specified-requirements.md	eos-ort03-norm	长满以下各级 + 条目落位（见 §2.1）

这条链不是 §1.3 的形态轴。 §1.3 的概要/完整态说的是**同一棵树长到多深**；本链说的是**同一份材料加工到多细**。两轴的落点也不同：切分与澄清两态树形相同（澄清只清语义），原始与文本化两态**不产树**（一个尚未进 EOS 结构、一个只换载体）——五个加工形态里只有两处动结构：切分（长出第一级）与规范化（长满其下各级）。

2.0.2 链上长出的结构

文本化原始需求 —— 一份材料 = 一份文本化文档（载体形态，结构照搬原文）
└ 业务对象闭环 —— ① 由切分产出（属性：所属业务域、相关方角色 = 主责角色）

切分产出的「业务对象闭环」就是 §2.1 的 ② 场景业务级——切分不是另起一棵树，而是把 §2.1 的场景级先长出来；规范化再把该级上的条目补满。故本链到切分为止给出的只是 §2.1 的**场景级**，其余各级一律以 §2.1 为唯一权威，本节不重述。

同一个「业务对象闭环」在三处出现，是同一个东西的三个方面：§2.0 的切分单位 / §2.1.1 的 ② 场景业务 / §2.2.1 的 ④ E2E 场景业务。

2.1 相关方需求

本份产品数据是 §2.0 入口加工链的终点（规范化后的形态）；其上游各态见 §2.0。

2.1.1 相关方功能需求

相关方功能需求 —— 根 本份产品数据自身
└ 相关方角色 —— ① 需求提出的角色（本份数据的索引 / 分片轴）
└ 场景业务 —— ② 主责角色 × 业务对象闭环（FR-BIZ 的天然单元；= §2.2.1 的

④)

└ 需求条目 — ③ 该场景下的一条 规范化的相关方需求 (spr-XXX)

本树与 §2.1.2 同法——以本份产品数据自身为根、画它自己的分解；条目是叶、场景是条目的归集级、角色是索引轴。**条目止于场景级**：场景内不设“业务输出文档 / 文档开发任务节点”之类中间单元（一条需求总是“某角色对某业务对象做某段闭环的事”，再往下就是业务侧的展开，不属本份数据）。

2.1.2 相关方非功能需求

相关方非功能需求

- └ 一级类目 ——— ① 质量特性 / 环境适应性 / 可实现性（面向哪类非功能关注）
 - └ 非功能主题 ——— ② 该级类目下的主题容器（示例：系统性能、信息安全性、系统可靠性）
 - └ 具体需求 — ③ 某主题下的具体非功能需求（需求描述 + 指标口径/目标值）

2.2 业务流程

§2.2 的业务树名 (R / P × 产品索引) 见 §1.1 名表。

本节读法：§2.2 不是几棵并列的树，而是按「产品 × 视角」组织——每个产品各有 R、P 两棵（平台自身为 R₀ / P₀，各业务为 R_i / P_i）。

枝梢与叶的边界 (R_i / P_i 两棵业务树)：**业务输出文档 (⑤) = 枝梢** (PL5，概要的末级)，**其下一级 = 叶**——PL6-A 活动节点、PL6-B 正文条目两支并列。「任务」不占树位：一个输出文档有 0 或 1 个任务，任务的主要节点与条目一样是**枝梢的分解源判据展开项**（名称级预告，完整态在 04 定义成叶）。

2.2.1 角色履职业务流程 (R 树, i ≥ 1)

角色履职业务

- └ 业务域 ——— ① 归属域
 - └ 相关方角色 ——— ② 主责角色（业务对象生命周期主推动者）
 - └ 角色职责 — ③ 角色承担的一类职责范围
 - └ E2E场景业务 — ④ 一个业务对象闭环 (D 执行 / PCA 管理)
 - └ 业务输出文档 — ⑤ 个人级的端到端，流程/工单/计划/看板/进展 (PL5，枝梢)
 - └ 活动节点 / 正文条目 — ⑥ 叶 (PL6-A 活动：处理角色/动作/状态变迁；PL6-B 条目：名称/定义要素/生命周期)

2.2.2 产品开发业务流程 (P 树, i ≥ 1)

输出产品业务

- └ 业务域 ——— ① 归属域
 - └ 产品类型 ——— ② 部门级的端到端业务输出
 - └ 产品构件类型 — ③ 科室级的端到端业务输出
 - └ 构件开发WBS (包) — ④ 文档序列

- └ 构件产品数据 (= 业务输出文档) — ⑤ 个人级的端到端，流程/工单/计划/看板/进展 (PL5, 枝梢)
 - └ 活动节点 / 正文条目 — ⑥ 叶 (PL6-A 活动: 处理角色/动作/状态变迁; PL6-B 条目: 名称/定义要素/生命周期)

2.2.3 引擎开发业务流程 (P 树, i = 0)

BP 产品树 (单分支—只有配置单元)

- └ 业务域 ———— ① L1 业务域 (IT与信息化)
 - └ 产品类型 ———— ② L2 产品类型 (EOS平台)
 - └ 构件 ———— ③ L3 构件 (引擎, 如 流程引擎 @engine-flow)
 - └ WBS ———— ④ L4 WBS (目录)
 - └ 配置单元 ———— ⑤ L5 配置单元 (CU, @xx-cu-zzz)
 - └ 配置信息组 — ⑥ L6 枝梢节点
 - └ 用例场景类操作 ———— ⑦ L7 叶节点

【待裁】P₀ 的级数与 91 的 PL 编号差一级，与本树“有无 WBS 层”是同一个待裁项——94 本树: ③ 构件 / ④ WBS(L4) / ⑤ 配置单元(L5) / ⑥ 配置信息组(L6) / ⑦ 叶(L7); 91 §3.1/§10.4/§10.6 (均称其为「A1 引擎产品的 PL 树」): **无 WBS** ("仅 A1 树以 CU 直接占据 PL4 位置")、CU = PL4、配置信息组 = PL5 **末级、无 PL6**。两说不能同时成立，且**不能拆开裁** (删掉“叶”这一级会让 ⑥ 的 L 标号跟着位移)，故本树本轮保持不动，整体并入该待裁项一次裁定。

2.3 系统需求

2.3.1 系统功能需求

SR 系统功能需求树

- └ 信息系统 ———— ① 承接业务流程的信息系统——业务流程可能被不同的信息系统承接，故先按承接系统分组 (可有多)
- └ 系统用例功能簇 — ② 对系统用例功能类的再聚类
 - └ 系统用例功能类 — ③ 按照某个维度对系统用例功能的聚类
 - └ 系统用例功能 — ④ 实现同一功能的不同实现方式:
 - └ 用例场景功能 — ⑤ 将用例场景 (用户视角) 转换为系统视角的功能
 - └ IPO功能 — ⑥ 系统对输入的响应

注 • ③ 的聚类维度 = 系统职责: ③ 系统用例功能类按**系统职责**聚类——回答“这个系统用例功能在系统里主要负责什么、由哪类机制承接”，**不是**“它属于哪个业务域、哪个业务对象”。② 是 ③ 的再聚类 (同一维度、粗一档)。① 信息系统只做“哪套系统承接业务”的分群，不替代职责聚类。

2.3.2 系统非功能需求

SR 系统非功能需求树

- └ 信息系统 ———— ① 承接系统 (与 §2.3.1 同一棵，非功能需求同样按承接系统挂)
 - └ 一级类目 — ② 质量特性 / 环境适应性 / 可实现性 (面向哪类非功能关注)

- └ 非功能主题 — ③ 该级类目下的主题容器（示例：系统性能、信息安全性、系统可靠性）
 - └ 具体系统非功能需求 — ④ 某主题下的 SysReq-NFR 末级（量化指标+统计口径+验证方法）

2.4 平台构成

- 企业信息系统 ——— 根 企业整体的信息系统（目标语境）
 - └ 信息系统 ——— ① 一个具体信息系统（EOS 平台=企业运营体系的信息化形态）
 - └ 模型 ——— ② 模型（目录性质：对应代码的文件夹）
 - └ 前后端组件 — ③ 枝梢节点（前端组件 / 后端组件）

2.5 配置方案

【待定义】结构树待定义——旧平台对象（功能表单 / 流程定义 ...）的产物归属在此收口。

2.6 前后端组件方案

2.6.1 前端组件方案

【待定义】结构树待定义。

2.6.2 后端组件方案

【待定义】结构树待定义。

变更记录

日期	版本	说明
2026-09-10	v1.14	\$2.2 树名改为「两视角族 × 产品索引」 （人类定）——R 树=角色履职视角（↔SL）、P 树=产品开发视角（↔PL）；i=0 平台本身、i≥1 基于平台引擎构建的各业务。\$1.1 原「三棵树」名表（A1/A2/Bn）作废，改为矩阵（R ₀ 【待确认】/ P ₀ = \$2.2.3、R _i = \$2.2.1 / P _i = \$2.2.2）；补两条口径（"所指产品的 i"与 SL0 / PL0 的层级 0 不同义；下标形式与 91 约束编号 R1~R4 相区分）；\$2.2 导语改「产品 × 视角」读法、三节标题与 \$1.4 承接表随迁
2026-09-10	v1.13	按 biz 链现有模型校正三处：① \$2.1.1 改画为 01 自身 的分解（相关方功能需求 → 相关方角色 → 场景业务 → 需求条目），原 A2 镜像树删除；② \$1.4 承接通则由「上游叶 → 下游枝梢」改为「上游最深可用原子 → 下游枝干/枝梢（以语义定级）」，补"两形态之间不是承接"与两条承接实例；③ \$2.2.1/\$2.2.2 末两级并为一级（⑤ 业务输出文档=枝梢 / ⑥ 活动节点·正文条目=叶），\$2.2 补枝梢—叶边界注、\$2.2.3 加【待裁】注（A1 级数并入 WBS 待裁项）；\$2.0.2 「\$2.1 树的根」改「\$2.1 的 ② 级」
2026-09-10	v1.12	新增 \$2.0 原始需求（入口加工链） ——五态表 + 链上长出的结构（切分产出的业务对象闭环= \$2.1 树的根）；\$1.1 清单表补原始需求加工链四行（† 注：同一份产品数据的四个加工形态）+ 首句改述；\$2.1 补回指 \$2.0 的注；\$二 导语「六份」改「各份」

日期	版本	说明
2026-09-10	v1.11	§1.1 清单表「规范化的相关方需求」→「相关方需求」（与 §2.1 节名一致）；表下注补一句对 91 全称的对齐
2026-09-10	v1.10	§1.1 平台构成拆为两行：平台构成概要（07） / 平台构成方案（文档待定义，新增待建）；例外注由三处改为两处
2026-09-10	v1.9	§1.1 清单表「形态」列并入「产品数据」列（业务流程概要 / 业务流程方案...），形态与待裁例外改由表下注说明
2026-09-10	v1.8	§1.1 标题简化为「EOS 的产品数据清单」
2026-09-10	v1.7	原 §二§七 六章合并为一章六节（§2.1§2.6）；外引联动 91 v5.56 / 05 v3.9 / 01-pl4eos-spec v2.2
2026-09-10	v1.6	§1.2 升格为「本文各树凭什么成立：三个前提」（条目化 / 树形化 / N:1 分配）
2026-09-10	v1.5	按 95 号报告落四条：新增树的成立条件、补叶的判据、补承接依据与反误读注；93 → v0.10
2026-09-10	v1.4	§1.1 改以产品数据清单开篇（以文档为粒度）
2026-09-10	v1.3	第一章按标题重构（清单 / 形态 / 承接）
2026-09-10	v1.2	第一章标题正名：各树对应关系 → 产品数据及其形态
2026-09-10	v1.1	章号自「一」起排（原 §〇 → §一，其余顺次后移）；外引联动 91 v5.55 / 93 v0.9 / 05 v3.8 / 01-pl4eos-spec v2.1
2026-09-10	v1.0	补首版版本行与变更记录；新增级语义与承接规则